Event loop & concurrency	Explain microtasks vs macrotasks, `setTimeout`/ `setImmediate`, and the difference between them.	Debug asynchronous order issues, explain the call stack and task queues.	Architect concurrency models using Web Workers, `SharedArrayBuffer`, and atomics for shared memory.

Having covered asynchronous behavior, we can now explore interview questions that showcase this topic.

Question: "What is event delegation in JavaScript?"

Analysis: This question assesses a candidate's understanding of efficient event handling in JavaScript, particularly in scenarios with multiple or dynamically generated elements. It evaluates knowledge of the DOM event model, event bubbling, and practical patterns for optimizing performance and maintainability in web applications.

Answer: Event delegation is a pattern in JavaScript where a single event listener is attached to a parent element instead of adding individual listeners to multiple child elements. This technique leverages event bubbling—a process in the DOM where an event triggered on a child element first runs its own event handlers, then successively triggers handlers on each of its ancestor elements up through the DOM tree, all the way to the outermost element or document object. Event delegation improves performance by reducing the number of event listeners, simplifies code maintenance, and automatically supports dynamically added elements without needing to attach new listeners. This approach is especially effective for managing events in dynamic lists or tables.

Suppose there is a list of items, as shown in listing 3.9.

Listing 3.9 A list of items

```
<ul id="myList">
  <li>Item 1</li>
```